

1.

Para los siguientes algoritmos de scheduling:

- FCFS (First Come First Served)
- SJF (Shortest Job First)
- Round Robin
- Prioridades

a. Explique su funcionamiento mediante un ejemplo.

b. ¿Alguno de ellos cuentan con parámetros para su funcionamiento? Identifique y enunciarlos

c. Cual es el más adecuado según los tipos de procesos y/o SO.

d. Cite ventajas y desventajas de su uso.

e. Defina Tiempo de retorno (TR) y Tiempo de espera (TE) para un proceso.

f. Defina Tiempo Promedio de Retorno (TPR) y Tiempo promedio de espera (TPE) para un lote de procesos.

g. Defina tiempo de respuesta

a.

Asumimos un lote de 3 procesos que llegan en el instante $t = 0$ con las siguientes ráfagas de CPU:

- P1: 10ms
- P2: 4ms
- P3: 2ms
- **FCFS (First Come First Served):** No apropiativo. Asigna la CPU estrictamente por orden de llegada¹.
 - *Ejecución:* P1 ejecuta (0-10ms), luego P2 (10-14ms), y finalmente P3 (14-16ms).
- **SJF (Shortest Job First):** No apropiativo. Selecciona el proceso con la ráfaga de CPU más corta disponible.
 - *Ejecución:* P3 ejecuta (0-2ms), luego P2 (2-6ms), y finalmente P1 (6-16ms).
- **Round Robin (Turno Rotativo):** Apropiativo. Cada proceso recibe un intervalo de tiempo máximo de CPU llamado **Quantum**; al expirar este tiempo, se realiza un cambio de contexto² y el proceso vuelve al final de la cola.
 - *Ejecución (Quantum = 3ms):* P1 (0-3) → P2 (3-6) → P3 (6-8, termina) → P1 (8-11) → P2 (11-12, termina) → P1 (12-16, termina).
- **Prioridades:** Apropiativo o no apropiativo. Asigna la CPU al proceso con mayor rango de prioridad numérica³.
 - *Ejecución (Prioridad: P2 > P3 > P1):* P2 ejecuta (0-4ms), luego P3 (4-6ms), y finalmente P1 (6-16ms).

b.

- **FCFS:** No requiere parámetros dinámicos de planificación (utiliza una cola estándar FIFO).
- **SJF:** Requiere la duración (o estimación matemática) de la próxima ráfaga de CPU de cada proceso.
- **Round Robin:** Requiere definir la duración del Quantum de tiempo (intervalo de asignación de CPU) y el ciclo de interrupción por hardware del reloj del sistema.
- **Prioridades:** Requiere asociar un valor numérico de prioridad a cada proceso (en GNU/Linux se modifica mediante llamadas al sistema como *nice*).

c.

- **Sistemas por lotes (Batch) sin interacción:** FCFS y SJF son los más adecuados porque minimizan los cambios de contexto no productivos para el procesador.
- **Sistemas interactivos y de tiempo compartido (escritorio/servidores):** Round Robin y Prioridades son los adecuados. Permiten asegurar tiempos de respuesta rápidos y proporcionalidad ante las acciones del usuario.

d.

Algoritmo	Ventajas	Desventajas
FCFS	Simple de implementar. Sin sobrecarga por cálculo de planificación.	Efecto Convoy: Procesos cortos esperan demasiado detrás de procesos largos.
SJF	Provee el tiempo de espera promedio mínimo garantizado.	Dificultad para predecir ráfagas futuras. Riesgo de inanición (starvation) para procesos largos.
Round Robin	Distribución equitativa de CPU. Excelente tiempo de respuesta interactivo.	Sensible al tamaño del Quantum (mucha sobrecarga por cambio de contexto si es muy corto).
Prioridades	Permite dar preferencia a procesos críticos o de tiempo real.	Inanición para procesos de baja prioridad si entran continuamente tareas prioritarias.

e.

Tiempo de Retorno (TR / Turnaround Time): Intervalo de tiempo total transcurrido desde el momento en que un proceso ingresa a la cola de listos (*Ready*) hasta el instante en que finaliza completamente su ejecución (*Exit*).

$$TR = \text{Tiempo de Finalización} - \text{Tiempo de Llegada}$$

Tiempo de Espera (TE / Waiting Time): Sumatoria de todos los períodos de tiempo que un proceso pasa dentro de la cola de listos esperando que el planificador le asigne el uso de la CPU.

$$TE = TR - \text{Tiempo de Ráfaga de CPU}$$

f.

Tiempo Promedio de Retorno (TPR): Promedio de los tiempos de retorno individuales del lote.

$$TPR = \frac{\sum_{i=1}^N TR_i}{N}$$

Tiempo Promedio de Espera (TPE): Promedio de los tiempos de espera individuales del lote.

$$TPE = \frac{\sum_{i=1}^N TE_i}{N}$$

g.

Tiempo de Respuesta: Intervalo de tiempo transcurrido desde que un proceso ingresa a la cola de listos (o se realiza una petición interactiva) hasta que el procesador comienza a ejecutarlo por primera vez y produce su primera respuesta o salida.

Dado el siguiente lote de procesos en el que todos arriban al sistema en el instante 0 (cero):

- Realice los diagramas de Gantt según los siguientes algoritmos scheduling:
 - FCFS (First Come, First Served)
 - SJF (Shortest Job First)
 - Round Robin con quantum = 4
- Para cada algoritmo calcule el TR y TE para cada job así como el TPR y el TPE.
- En base a los tiempos calculados compare los diferentes algoritmos.

JOB	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	TR	TE		
1	7	>1	2	3	4	5	6	7<																																								7	0			
2	15	>							1	2	3	4	5	6	7	8	9	10	11	12	13	14	15<																												22	7
3	12	>																						1	2	3	4	5	6	7	8	9	10	11	12<															34	22	
4	4	>																																			1	2	3	4<											38	34
5	9	>																																																	47	38
FCFS																																																			29.6	20.2

JOB	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	TR	TE			
1	7	>			1	2	3	4	5	6	7<																																									11	4
2	15	>																																																		47	32
3	12	>																						1	2	3	4	5	6	7	8	9	10	11	12	13	14	15<												32	20		
4	4	>1	2	3	4<																																														4	0	
5	9	>											1	2	3	4	5	6	7	8	9<																														20	11	
SJF																																																			22.8	13.4	

JOB	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	TR	TE
1	7	>1	2	3	4																																													

ALGORITMO	TPR	TPE
FCFS	22.8	13.4
SJS	29.6	20.2
RR Q=4	34.6	25.2

Dado el siguiente lote procesos:

Job	Llegada	Unidades de CPU
1	0	4
2	2	6
3	3	4
4	6	5
5	8	2

- a. Realice los diagramas de Gantt según los siguientes algoritmos de scheduling:
 - FCFS (First Come, First Served)
 - SJF (Shortest Job First)
 - Round Robin con quantum = 1
 - Round Robin con quantum = 6
- b. Para cada algoritmo calcule el TR y TE para cada job así como el TPR y el TPE.
- c. En base a los tiempos calculados compare los diferentes algoritmos.
- d. En el algoritmo Round Robin, qué conclusión se puede sacar con respecto al valor del quantum.
- e. ¿Para el algoritmo Round Robin, en qué casos utilizará un valor de quantum alto y qué ventajas y desventajas obtendría?

a y b.

[illegible][illegible]

JOB	LLEGADA	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20		TR	TE
1	0	4	>1	2	3		4<																		5	1
2	2	6			>	1			2			3				4				5		6<			18	12
3	3	4				>		1		2			3				4<								12	8
4	6	5							>		1				2				3		4		5<		15	10
5	8	2									>			1				2<							8	6
RR	Q=1	QUEU	1	1	1	2	1	3	2	3	4	2	3	5	4	2	3	5	4	2	4	2	4		11.6	7.4

[illegible]

c.

ALGORITMO	TPR	TPE
FCFS	11.6	7.4
SJS	9.8	5.6
RR Q=1	9.8	5.6
RR Q=6	8.4	4.2

d.

Si el quantum es muy chico, los procesos se alternan muy seguido. Eso provoca que el TPR y el TPE empeoren. Si el quantum es muy grande, el comportamiento se asemeja a FCFS

e.

Se usa Quantum grande cuando es importante terminar los procesos eficientemente y no la velocidad de respuesta.

Una ventaja es menor context switches

Una desventaja es que los procesos cortos pueden esperar mucho a los procesos largos.

4.

Una variante al algoritmo SJF es el algoritmo SJF apropiativo o SRTF (Shortest Remaining Time First):

a) Realice el diagrama de Gantt para este algoritmo según el lote de trabajos del ejercicio 3.

b. ¿Nota alguna ventaja frente a otros algoritmos?

a.

JOB	LLEGADA	CPU	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20		TR	TE
1	0	4	>1	2	3	4<																			4	0
2	2	6			>													1	2	3	4	5	6<		19	13
3	3	4				>	1	2	3	4<															5	1
4	6	5								>			1	2	3	4	5<								9	4
5	8	2									>1	2<													2	0
SRTF		QUEU	1		2	3			4		5														7.80	3.60

b.

SRTF suele ser más eficiente que SJF en entornos donde los procesos llegan en distintos momentos (arribos intermitentes).

Si todos los procesos llegan al mismo tiempo, ambos algoritmos funcionan igual de rápido.

5.

Suponga que se agregan las siguientes prioridades al lote de procesos del ejercicio 3, donde un menor número indica mayor prioridad:

Job	Prioridad
1	3
2	4
3	2
4	1
5	2

- Realice el diagrama de Gantt correspondiente al algoritmo de planificación por prioridades según las variantes:
 - No Apropiativa
 - Apropiativa
- Calcule el TR y TE para cada job así como el TPR y el TPE.
- ¿Nota alguna ventaja frente a otros algoritmos? ¿Bajo qué circunstancias lo utilizaría y ante qué situaciones considera que la implementación de prioridades podría no ser de mayor relevancia?

a y b.

JOB	LLEGADA	CPU	PRIORIDAD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	TR	TE
1	0	4	3	>1	2	3	4<																		4	0
2	2	6	4			>													1	2	3	4	5	6<	19	13
3	3	4	2				>	1	2	3	4<														5	1
4	6	5	1							>		1	2	3	4	5<									7	2
5	8	2	2									>					1	2<							7	5
			QUEU 1	4																						
			QUEU 2	3	5																					
			QUEU 3	1																						
			QUEU 4	2																						
NO APROPIATIVO																									8.40	4.20

JOB	LLEGADA	CPU	PRIORIDAD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	TR	TE	
1	0	4	3	>1	2	3												4<							15	11	
2	2	6	4			>												1	2	3	4	5	6<	19	13		
3	3	4	2				>1	2	3						4<										9	5	
4	6	5	1							>1	2	3	4	5<											5	0	
5	8	2	2									>				1	2<								6	4	
			QUEU 1	4																							
			QUEU 2	3	3	5																					
			QUEU 3	1	1																						
			QUEU 4	2																							
APROPIATIVO																										10.80	6.60

c.

- Ventajas:** Prioriza tareas críticas/urgentes y reduce el tiempo de respuesta de los procesos interactivos.
- Cuando usarlo:** En sistemas con procesos heterogéneos (interactivos vs. cálculo intensivo) o de tiempo real donde la rapidez de respuesta es clave.
- Cuando NO es relevante:** En sistemas con trabajos homogéneos (todos los procesos son igual de importantes) o si se busca equidad estricta entre usuarios (donde conviene Round Robin).

6. Inanición (Starvation)

a. ¿Qué significa?

b. ¿Cuál/es de los algoritmos vistos puede provocarla?

c. ¿Existe alguna técnica que evite la inanición para el/los algoritmos mencionados en el inciso b?

a.

La inanición (o *starvation*) es una situación en la que un proceso que se encuentra en la cola de listos (*ready queue*) es postergado de forma indefinida y nunca (o tras un tiempo excesivo) obtiene el uso de la CPU. Esto ocurre porque el algoritmo de planificación favorece continuamente a otros procesos, violando la meta de equidad (*fairness*), que busca otorgar una parte justa del procesador a cada tarea.

b.

1. SJF (*Shortest Job First*) y SRTF (*Shortest Remaining Time First*):

- **Razón:** Otorgan prioridad a los procesos con las ráfagas de CPU más cortas. Si existe un flujo constante de procesos cortos, un proceso con una ráfaga larga de CPU (*CPU-bound*) quedará postergado indefinidamente en la cola.

2. Planificación por Prioridades Estáticas:

- **Razón:** Si a cada proceso se le asigna una prioridad fija al crearse y el sistema recibe continuamente procesos de prioridad más alta, los procesos de baja prioridad nunca serán seleccionados por el planificador de corto plazo (*Short Term Scheduler*).

3. Colas Multinivel con Prioridad Estricta:

- **Razón:** Si el planificador le otorga prioridad absoluta a una cola (por ejemplo, la cola de procesos interactivos), los procesos en las colas de menor prioridad (como los procesos *batch*) sufrirán inanición mientras la cola de mayor prioridad no se vacíe por completo.

c.

- **Envejecimiento (*Aging*):** Es la técnica principal para solucionar la inanición en algoritmos basados en prioridades, SJF y SRTF. Consiste en aumentar de forma progresiva la prioridad de un proceso a medida que transcurre el tiempo que permanece esperando en la cola de listos. De este modo, con el paso del tiempo, la prioridad del proceso relegado aumenta lo suficiente como para superar a los procesos de reciente llegada y obtener la CPU.
- **Reparto por Franjas de Tiempo (*Time-Slicing*) en Colas Multinivel:** En lugar de prioridad estricta entre colas, se asigna a cada cola un porcentaje fijo del tiempo total de la CPU (por ejemplo, 80% para la cola interactiva y 20% para la cola batch), garantizando que las colas de menor prioridad reciban atención periódicamente.

7. Los procesos, durante su ciclo de vida, pueden realizar operaciones de I/O como lecturas o escrituras a disco, cintas, uso de impresoras, etc. El Kernel mantiene para cada dispositivo, que se tiene en el equipo, una cola de procesos que espera por la utilización del mismo (al igual que ocurre con la Cola de Listos y la CPU, ya que la CPU es un dispositivo más). Cuando un proceso en ejecución realiza una operación de I/O el mismo es expulsado de la CPU y colocado en la cola correspondiente al dispositivo involucrado en la operación. El Kernel dispone también de un “I/O Scheduling” que administra cada cola de dispositivo a través de algún algoritmo (FCFS, Prioridades, etc.). Si al colocarse un proceso en la cola del dispositivo, la misma se encuentra vacía el mismo será atendido de manera inmediata, caso contrario, deberá esperar a que el Kernel lo seleccione según el algoritmo de scheduling establecido. Los mecanismos de I/O utilizados hoy en día permiten que la CPU no sea utilizada durante la operación, por lo que el Kernel puede ejecutar otro proceso que se encuentre en espera una vez que el proceso bloqueado por la I/O se coloca en la cola correspondiente. Cuando el proceso finaliza la operación de I/O el mismo retorna a la cola de listos para competir nuevamente por la utilización de la CPU. Para los siguientes algoritmos de Scheduling:

➤ FCFS

➤ Round Robin con quantum = 2

Y suponiendo que la cola de listos de todos los dispositivos se administra mediante FCFS, realice los diagramas de Gantt según las siguientes situaciones:

a. Suponga que al lote de procesos del ejercicio 5 se agregan las siguientes operaciones de entrada salida:

Job	I/O (Recurso,Instante,Duración)
1	(R1, 2, 1)
2	(R2, 3, 1) (R2, 5, 2)
4	(R3, 1, 2) (R3, 3, 1)

b. Suponga que al lote de procesos del ejercicio 5 se agregan las siguientes operaciones de entrada salida:

Job	I/O (Recurso,Instante,Duración)
1	(R1, 2, 3) (R1, 3, 2)
2	(R2, 3, 2)
3	(R2, 2, 3)
4	(R1, 1, 2)

a.

[illegible][illegible]

b.

[illegible][illegible]

8. Algunos algoritmos pueden presentar ciertas desventajas cuando en el sistema se cuenta con procesos ligados a CPU y procesos ligados a entrada salida. Analice las mismas para los siguientes algoritmos:

a. Round Robin

b. SRTF (Shortest Remaining Time First)

a.

- **Mecanismo:** Asigna la CPU a cada proceso en la cola de listos durante un tiempo máximo determinado (*quantum* o *time slice*).
- **Desventajas/Problemas:**
 1. **Penalización e inequidad hacia los procesos I/O-bound:** Los procesos ligados a E/S ejecutan una ráfaga de CPU muy corta, hacen su solicitud de E/S y liberan la CPU voluntariamente antes de agotar su *quantum*. Mientras realizan la E/S, la CPU se le otorga a un proceso CPU-bound, el cual suele consumir su *quantum* completo.
 2. **Subutilización de dispositivos de E/S:** Cuando el proceso I/O-bound termina su operación de E/S y regresa a la cola de listos, se ubica al final de la misma. Debe esperar a que todos los procesos CPU-bound previos consuman sus *quantums* completos antes de volver a ejecutar. Esto provoca que los dispositivos de E/S queden ociosos durante largos períodos esperando a que el proceso I/O-bound vuelva a pedir E/S, reduciendo la eficiencia general del sistema.

b.

- **Mecanismo:** Algoritmo apropiativo que asigna la CPU al proceso con el menor tiempo restante estimado de ráfaga de CPU.
- **Desventajas/Problemas:**
 1. **Inanición (*Starvation*) de procesos CPU-bound:** Dado que los procesos I/O-bound se caracterizan por ráfagas de CPU muy breves, SRTF siempre les dará máxima prioridad sobre los procesos CPU-bound. Si existe un flujo constante de ráfagas cortas de E/S, los procesos ligados a CPU nunca obtendrán la CPU o sufrirán demoras excesivas (*starvation*).
 2. **Dificultad de estimación y overhead:** SRTF requiere conocer o predecir la duración de la próxima ráfaga de CPU de cada proceso. Estimar esto con precisión genera una sobrecarga (*overhead*) en el sistema operativo.

10. Suponga que un Kernel utiliza un algoritmo de VRR para planificar procesos. Para ello, el quantum es representado por un contador, que es decrementado en 1 unidad cada vez que ocurre una interrupción de reloj. ¿Bajo este esquema, puede suceder que el quantum de un proceso nunca llegue a 0 (cero)? Justifique su respuesta.

Sí, puede suceder.

En un esquema de planificación con interrupción de reloj, la CPU utiliza un contador que se asigna como *quantum* y se decrementa con cada *tick* de reloj. Sin embargo, el *quantum* de un proceso puede no llegar a cero por las siguientes razones:

1. **Solicitud de Entrada/Salida o bloqueo voluntario:** Un proceso en estado de ejecución (*running*) conserva la CPU hasta que expira su franja de tiempo (*quantum*), hasta que finaliza, o **hasta que requiere realizar una operación de E/S o esperar un evento**. Si el proceso es ligado a E/S (*I/O-bound*) o realiza una llamada al sistema bloqueante (*read*, *write*, *wait*, *sleep*) antes de agotar su tiempo, abandona voluntariamente la CPU antes de que el contador llegue a cero.
2. **Mecanismo del algoritmo VRR (Virtual Round Robin):** En VRR, los procesos que liberan la CPU por E/S pasan a una cola auxiliar prioritaria. Al volver a ejecutarse, reciben únicamente el remanente (tiempo restante) de su *quantum* anterior. Si el proceso realiza una nueva operación de E/S rápidamente dentro de ese tiempo parcial, liberará la CPU nuevamente antes de que el contador decremente a 0.
3. **Terminación del proceso:** Si el proceso completa su trabajo y ejecuta la llamada de finalización (*exit*) antes de consumir la totalidad del tiempo otorgado, la CPU es liberada de inmediato sin llegar a decrementar el contador a cero.

11. El algoritmo SJF (y SRTF) tiene como su principal problema su implementación, dada la dificultad de conocer la duración de la próxima ráfaga de CPU. Es posible realizar una estimación de la próxima, utilizando la media de las ráfagas de CPU para cada proceso. Así, por ejemplo, podemos tener la siguiente fórmula (1):

$$S_{n+1} = \frac{1}{n}T_n + \frac{n-1}{n}S_n$$

Donde:

T_i = duración de la ráfaga de CPU i-ésima del proceso.

S_i = valor estimado para el i-ésimo caso

S_1 = valor estimado para la primera ráfaga de CPU. No es calculado.

a. Suponga un proceso cuyas ráfagas de CPU reales tienen como duración: 6, 4, 6, 4, 13, 13, 13. Calcule qué valores se obtendrían como estimación para las ráfagas de CPU del proceso si se utiliza la fórmula 1, con un valor inicial estimado de $S_1=10$. La fórmula 1 les da el mismo peso a todos los casos (siempre calcula la media). Es posible reescribirla permitiendo darle un peso mayor a los casos más recientes y menor a casos viejos (o viceversa). Se plantea la siguiente fórmula (2):

$$S_{n+1} = \alpha T_n + (1 - \alpha)S_n$$

Con $0 < \alpha < 1$.

b. Analice para qué valores de α se tienen en cuenta los casos más recientes.

c. Para la situación planteada en a) calcule qué valores se obtendrían si se utiliza la fórmula 2 con $\alpha = 0,2$; $\alpha = 0,5$ y $\alpha = 0,8$.

d. Para todas las estimaciones realizadas en a y c ¿Cuál es la que más se asemeja a las ráfagas de CPU reales del proceso?

a.

$$\begin{aligned} S_1 &= 10 \\ S_2 &= (1/1)6 + ((1-1)/1)10 = 6 \\ S_3 &= (1/2)4 + ((2-1)/2)6 = 5 \\ S_4 &= (1/3)6 + ((3-1)/3)5 = 5.33 \\ S_5 &= (1/4)4 + ((4-1)/4)5.33 = 4.99 \\ S_6 &= (1/5)13 + ((5-1)/5)4.99 = 6.592 \\ S_7 &= (1/6)13 + ((6-1)/6)6.592 = 7.66 \\ S_8 &= (1/7)13 + ((7-1)/7)7.66 = 8.42 \end{aligned}$$

b.

Cuanto más cercano a 1 sea α , mayor peso tiene el valor real más reciente ($T(n)$) en la nueva estimación, haciendo que el sistema reaccione más rápido a cambios recientes en el comportamiento del proceso. Cuanto más cercano a 0, mayor peso tiene la historia acumulada ($S(n)$), haciendo que la estimación cambie más lentamente y sea más estable frente a variaciones puntuales.

C.

$\alpha = 0,2$

$$S_1 = 10$$

$$S_2 = 0.2 \times 6 + (1 - 0.2) \times 10 = 9.2$$

$$S_3 = 0.2 \times 4 + (1 - 0.2) \times 9.2 = 8.16$$

$$S_4 = 0.2 \times 6 + (1 - 0.2) \times 8.16 = 7.728$$

$$S_5 = 0.2 \times 4 + (1 - 0.2) \times 7.728 = 6.9824$$

$$S_6 = 0.2 \times 13 + (1 - 0.2) \times 6.9824 = 8.18592$$

$$S_7 = 0.2 \times 13 + (1 - 0.2) \times 8.18592 = 9.148736$$

$$S_8 = 0.2 \times 13 + (1 - 0.2) \times 9.148736 = 9.9189888$$

$\alpha = 0,5$

$$S_1 = 10$$

$$S_2 = 0.5 \times 6 + (1 - 0.5) \times 10 = 8$$

$$S_3 = 0.5 \times 4 + (1 - 0.5) \times 8 = 6$$

$$S_4 = 0.5 \times 6 + (1 - 0.5) \times 8 = 7$$

$$S_5 = 0.5 \times 4 + (1 - 0.5) \times 7 = 5.5$$

$$S_6 = 0.5 \times 13 + (1 - 0.5) \times 5.5 = 9.25$$

$$S_7 = 0.5 \times 13 + (1 - 0.5) \times 9.25 = 11.125$$

$$S_8 = 0.5 \times 13 + (1 - 0.5) \times 11.125 = 12.0625$$

$\alpha = 0,8$

$$S_1 = 10$$

$$S_2 = 0.8 \times 6 + (1 - 0.8) \times 10 = 6.8$$

$$S_3 = 0.8 \times 4 + (1 - 0.8) \times 6.8 = 4.56$$

$$S_4 = 0.8 \times 6 + (1 - 0.8) \times 4.56 = 5.712$$

$$S_5 = 0.8 \times 4 + (1 - 0.8) \times 5.712 = 4.3424$$

$$S_6 = 0.8 \times 13 + (1 - 0.8) \times 4.3424 = 11.26848$$

$$S_7 = 0.8 \times 13 + (1 - 0.8) \times 11.26848 = 12.653696$$

$$S_8 = 0.8 \times 13 + (1 - 0.8) \times 12.653696 = 12.9307392$$

12. Colas Multinivel

Actualmente los algoritmos de planificación vistos se han ido combinando para formar algoritmos más eficientes. Así surge el algoritmo de Colas Multinivel, donde la cola de procesos listos es dividida en varias colas, teniendo cada una su propio algoritmo de planificación.

a. Suponga que se tiene dos tipos de procesos: Interactivos y Batch.

Cada uno de estos procesos se coloca en una cola según su tipo. ¿Qué algoritmo de los vistos utilizará para administrar cada una de estas colas?

A su vez, se utiliza un algoritmo para administrar cada cola que se crea. Así, por ejemplo, el algoritmo podría determinar mediante prioridades sobre qué cola elegir un proceso.

b. Para el caso de las dos colas vistas en a: ¿Qué algoritmo utilizaría para planificarlas?

a.

La elección del algoritmo interno para cada cola depende de la naturaleza y las metas del tipo de proceso que contiene:

- **Cola de Procesos Interactivos:**
 - **Algoritmo recomendado:** Round Robin (RR) (o SRTF).
 - **Justificación:** Los procesos interactivos requieren dar respuesta rápida a las peticiones del usuario (minimizar el tiempo de respuesta). Para estos procesos son necesarios algoritmos apropiativos (*preemptive*), de manera que un proceso no se apropie indefinidamente de la CPU. Round Robin garantiza que todos los procesos interactivos reciban una franja de tiempo (*quantum*) de forma equitativa.
- **Cola de Procesos Batch (por lotes):**
 - **Algoritmo recomendado:** FCFS (First Come First Served) o SJF (Shortest Job First).
 - **Justificación:** Los trabajos batch no tienen un usuario esperando en tiempo real ante la terminal. Sus metas principales son maximizar la cantidad de trabajos completados por unidad de tiempo (*throughput*), mantener ocupada la CPU y minimizar el tiempo de retorno. Por ello, se benefician de algoritmos no apropiativos como FCFS o SJF.

b.

Para determinar qué cola obtiene el uso de la CPU en un momento dado, se utilizan habitualmente dos estrategias:

1. **Prioridad Estricta (con expropiación):**
 - Se le asigna prioridad absoluta a la cola de procesos interactivos por encima de la cola batch.
 - La CPU ejecutará procesos de la cola batch solo cuando la cola interactiva esté completamente vacía. Si llega un proceso interactivo

mientras se ejecuta un proceso batch, este último es interrumpido (*preempted*) para atender de inmediato la interacción del usuario.

2. Reparto de Franjas de Tiempo (Time-Slicing / Asignación por Porcentajes):

- Se asigna un porcentaje fijo del tiempo de procesador a cada cola (por ejemplo, 80% del tiempo de CPU para la cola interactiva y 20% para la cola batch).
- **Ventaja:** Previene el problema de inanición (*starvation*) de los trabajos batch en situaciones donde la cola de procesos interactivos presente un tráfico constante.

Se tiene el siguiente lote de procesos a ser procesados con sus respectivas operaciones de I/O:

Job	Llegada	CPU	I/O (rec,ins,dur)	Prioridad
1	0	9	(R1, 4, 2) (R2, 6, 3) (R1, 8, 3)	1
2	1	5	(R3, 3, 2) (R3, 4, 2)	2
3	2	5	(R1, 4, 1)	3
4	3	7	(R2, 1, 2) (R2, 5, 3)	2
5	5	5	(R1, 2, 3) (R3, 4, 3)	1

a. Asumiendo que NO hay apropiación entre los procesos.
b. Asumiendo que hay apropiación entre los procesos.

[illegible][illegible]

14. En el esquema de Colas Multinivel, cuando se utiliza un algoritmo de prioridades para administrar las diferentes colas los procesos pueden sufrir starvation. La técnica de envejecimiento se puede aplicar a este esquema, haciendo que un proceso cambie de una cola de menor prioridad a una de mayor prioridad, después de cierto periodo de tiempo que el mismo se encuentra esperando en su cola. Luego de llegar a una cola en la que el proceso llega a ser atendido, el mismo retorna a su cola original. Por ejemplo: Un proceso con prioridad 3 está en cola su cola correspondiente. Luego de X unidades de tiempo, el proceso se mueve a la cola de prioridad 2. Si en esta cola es atendido, retorna a su cola original, en caso contrario luego de sucederse otras X unidades de tiempo el proceso se mueve a la cola de prioridad 1. Esta última acción se repite hasta que el proceso obtiene la CPU, situación que hace que el mismo vuelva a su cola original. Para los casos a y b del ejercicio 15 realice el diagrama de Gantt considerando además que se tiene un envejecimiento de 4 unidades.

Caso a.

JOB	LLEGADA	CPU	E/S	PRIORIDAD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	TR	TE
1	0	9	(R1, 4, 2) (R2, 6, 3) (R1, 8, 3)	1		<1	2	3	4	R1	R1																											25	16
2	1	5	(R3, 3, 2) (R3, 4, 2)	2			<			1	2	3	R3	R3									4															26	21
3	2	5	(R1, 4, 1)	3				<																														27	22
4	3	7	(R2, 2, 2) (R2, 5, 3)	2					<								1	R2	R2																			30	23
5	5	5	(R1, 2, 3) (R3, 4, 3)	1																																		19	14
RR			QUANTUM = 4																																			25,4	21,2
				QUEU 1		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		
				QUEU 2		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		
				QUEU 3		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		

Caso b.

JOB	LLEGADA	CPU	E/S	PRIORIDAD	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	TR	TE
1	0	9	(R1, 4, 2) (R2, 6, 3) (R1, 8, 3)	1		<1	2	3	4	R1	R1																											25	16
2	1	5	(R3, 3, 2) (R3, 4, 2)	2			<			1																												26	21
3	2	5	(R1, 4, 1)	3				<																														27	22
4	3	7	(R2, 2, 2) (R2, 5, 3)	2					<																													33	26
5	5	5	(R1, 2, 3) (R3, 4, 3)	1																																		19	14
RR			QUANTUM = 4																																			26	19,8
				QUEU 1		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		
				QUEU 2		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		
				QUEU 3		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32		

15. La situación planteada en el ejercicio anterior, donde un proceso puede cambiar de una cola a otra, se la conoce como Colas Multinivel con Realimentación. Suponga que se quiere implementar un algoritmo de planificación que tenga en cuenta el tiempo de ejecución consumido por el proceso, penalizando a los que más tiempo de ejecución tienen. (Similar a la tarea del algoritmo SJF que tiene en cuenta el tiempo de ejecución que resta). Utilizando los conceptos vistos de Colas Multinivel con Realimentación indique qué colas implementaría, qué algoritmo usaría para cada una de ellas así como para la administración de las colas entre sí. Tenga en cuenta que los procesos no deben sufrir inanición.

1. Configuración de las Colas

Se implementan 3 colas de procesos listos:

- **Cola 0 (Q0 - Alta prioridad):** Algoritmo Round Robin con quantum chico ($q = 8$). Recibe todos los procesos nuevos y favorece a los procesos cortos e interactivos.
- **Cola 1 (Q1 - Prioridad media):** Algoritmo Round Robin con quantum mayor ($q = 16$).
- **Cola 2 (Q2 - Baja prioridad):** Algoritmo FCFS (o Round Robin con quantum amplio) para procesar los trabajos largos.

2. Administración entre las Colas

- **Prioridad Estricta Apropiativa (Preemptive Priority):** La CPU atiende primero a Q0. Solo si Q0 está vacía pasa a Q1, y si ambas están vacías ejecuta Q2. Si ingresa un proceso a una cola superior, desbanca al proceso actual de menor prioridad.

3. Criterio de Realimentación (Penalización)

- **Regla de descenso:** Todo proceso ingresa por **Q0**. Si agota completamente su quantum en Q0 sin finalizar ni bloquearse por E/S, se lo considera de uso intensivo de CPU (*CPU-bound*) y el sistema lo degrada a Q1. Si agota su quantum en Q1, desciende a Q2.
- **Premio por E/S:** Si un proceso libera la CPU voluntariamente por Entrada/Salida antes de agotar su quantum, mantiene su nivel de prioridad al regresar.

4. Mecanismo contra la Inanición (Starvation)

- **Envejecimiento (Aging):** Si un proceso permanece en Q1 o Q2 esperando en la cola durante más de un tiempo límite X sin recibir CPU, el sistema lo promociona automáticamente a la cola de prioridad inmediatamente superior.

16. A cuáles de los siguientes tipos de trabajos:

- cortos acotados por CPU
- cortos acotados por E/S
- largos acotados por CPU
- largos acotados por E/S

benefician las siguientes estrategias de administración:

- a. prioridad determinada estáticamente con el método del más corto primero (SJF).**
- b. prioridad dinámica inversamente proporcional al tiempo transcurrido desde la última operación de E/S.**

a.

- **Trabajos beneficiados:** Cortos acotados por CPU y cortos acotados por E/S.
- **Justificación:** El algoritmo SJF (*Shortest Job First*) selecciona y asigna la CPU al proceso que presenta la menor duración o la ráfaga de CPU más corta.
 - Los trabajos cortos acotados por CPU se benefician porque son atendidos rápidamente y completan su ejecución sin sufrir largas esperas.
 - Los trabajos cortos acotados por E/S presentan ráfagas de CPU muy breves antes de requerir E/S, por lo que bajo el criterio de SJF obtienen prioridad sobre cualquier proceso con ráfagas más largas.

b.

- **Trabajos beneficiados:** Cortos acotados por E/S y largos acotados por E/S.
- **Justificación:** Que la prioridad sea *inversamente proporcional* al tiempo transcurrido desde la última E/S implica que a menor tiempo desde que el proceso realizó E/S (es decir, cuanto más recientemente/frecuentemente haya solicitado una operación de E/S), mayor será su prioridad.
 - Esta estrategia favorece a todos los trabajos acotados por E/S (*I/O-bound*), ya que al recuperar la CPU tras una interrupción de E/S obtienen una prioridad alta inmediata.
 - Otorgar prioridad a los trabajos acotados por E/S permite que consuman su breve ráfaga de CPU y vuelvan rápidamente a bloquearse esperando E/S, manteniendo los dispositivos periféricos ocupados y maximizando el aprovechamiento general del sistema.

17. Analizar y explicar por qué si el quantum en Round-Robin se incrementa sin límite, el algoritmo de planificación se aproxima a FIFO.

Si el quantum (franja de tiempo) en el algoritmo Round-Robin (RR) se incrementa sin límite ($Q \rightarrow \infty$), el algoritmo se aproxima y equivale al algoritmo FIFO (First In, First Out / FCFS - First Come, First Served)

1. **Funcionamiento normal de Round-Robin:** Round-Robin mantiene a los procesos en una cola de listos con disciplina FIFO y les asigna la CPU durante un tiempo máximo determinado (*quantum*). Si un proceso no finaliza ni se bloquea por E/S antes de que expire dicho tiempo, es expulsado de la CPU mediante una interrupción de reloj y reubicado al final de la cola de listos.
2. **Efecto de un *quantum* tendiente a infinito:** Si el valor del *quantum* se vuelve extremadamente grande (mayor que la ráfaga de CPU más larga de cualquier proceso del sistema), la interrupción por fin de *quantum* jamás llegará a ejecutarse.
3. **Convergencia hacia FIFO / FCFS:** Al desaparecer la apropiación por expiración del *quantum*, los procesos se ejecutan de manera ininterrumpida hasta que finalizan o se bloquean voluntariamente para realizar una operación de Entrada/Salida. Dado que la selección de la CPU sigue el estricto orden de llegada a la cola de listos, la planificación se transforma en un esquema no apropiativo basado en orden de llegada, es decir, el algoritmo FIFO / FCFS.

18. Dado los siguientes programas en un pseudo-código que simula la utilización de llamadas al sistema para crear procesos en Unix/Linux, indicar qué mensajes se imprimirán en pantalla (sin importar el orden en que saldrían):

a. Caso 1

```
printf("hola")
x = fork ()
if x < 1 {
    execv("ls")
    printf("mundo")
    exit(0)
}
exit(0)
```

b. Caso 2

```
printf("hola")
x = fork ()
if x < 1 {
    execv("ps")
    printf("mundo")
    exit (0)
}
execv("ls")
printf("fin")
exit (0)
```

c. Caso 3

```
printf("Anda a rendir el Primer Parcial de Promo!")
newpid = fork()
if newpid == 0 {
    printf ("Estoy comenzando el Examen")
    execv("ps")
    printf ("Termine el Examen")
}
printf ("¿Como te fue?")
exit (0)
printf ("Ahora anda a descansar")
```

a.

- "hola"
- [Listado de archivos impreso por el comando ls]

b.

- "hola"
- [Listado de procesos impreso por el comando ps]
- [Listado de archivos impreso por el comando ls]

c.

- "Anda a rendir el Primer Parcial de Promo!"
- "Estoy comenzando el Examen"
- [Listado de procesos impreso por el comando ps]
- "¿Como te fue?"

19. Dado el siguiente programa en C, analice su objetivo sin necesidad de ejecutarlo. (tenga en cuenta el funcionamiento del iterador for en C para poder responder:

<https://ccia.ugr.es/~jfv/ed1/c/cdrom/cap4/cap43.htm>)

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main (void) {
    int c;
    pid_t pid;
    printf ("Comienzo.: \n ");
    for (c = 0; c < 3; c++) {
        pid = fork ();
    }
    printf ("Proceso \n");
    return 0;
}
```

a. ¿Cuántas líneas con la palabra “Proceso” aparecen al final de la ejecución de este programa?

b. ¿El número de líneas es el número de procesos que han estado en ejecución? Ejecute el programa y compruebe si su respuesta es correcta. Modifique el valor del bucle for y compruebe los nuevos resultados.

a.

Aparecen 8 líneas con la palabra "Proceso".

b.

Sí, el número de líneas impresas coincide exactamente con el número de procesos totales que han estado en ejecución (8 procesos). Dado que ningún proceso finaliza antes de tiempo, todos los procesos duplicados llegan al final del código y ejecutan la instrucción de impresión.

- **Comprobación al modificar el bucle for:** Si se modifica el bucle para cambiar la cantidad de iteraciones n , el número de líneas impresas responde a la fórmula 2^N :
 - **Para $c < 1$ (1 iteración):** $2^1 = 2$ procesos / 2 líneas.
 - **Para $c < 2$ (2 iteraciones):** $2^2 = 4$ procesos / 4 líneas.
 - **Para $c < 4$ (4 iteraciones):** $2^4 = 16$ procesos / 16 líneas.

20. Modifiquemos el programa anterior. Ahora, además de un mensaje, vamos a añadir una variable y antes de finalizar el programa vamos a mostrar el valor de la misma:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
int main (void) {
    int c;
    pid_t pid;
    int p = 0;
    printf ("Comienzo.: \n ");
    for (c = 0; c < 3; c++) {
        pid = fork ();
    }
    p++;
    printf ("Proceso %d\n ", p);
    return 0;
}
```

- a. ¿Qué valores se imprimirán en la consola?
- b. ¿Todas las líneas tendrán el mismo valor o algunas líneas tendrán valores distintos?
- c. ¿Cuál es el valor (o valores) que aparece? Compile y ejecute el programa y compruebe si su respuesta es correcta.
- d. Realice modificaciones al programa, por ejemplo: modifique el valor del bucle for, cambie el lugar dónde se incrementa la variable p, y compruebe los nuevos resultados.

a.

Se imprimirá en pantalla la línea Proceso 1 un total de 8 veces (una vez por cada uno de los 8 procesos que llegan a la instrucción printf).

b.

Todas las líneas tendrán exactamente el mismo valor. Ninguna línea mostrará un número diferente.

c.

Aparece únicamente el valor 1.

Justificación: Debido a la protección de memoria y la independencia del espacio de direcciones que otorga fork(), las modificaciones en las variables locales de un proceso

no afectan las variables de los demás procesos. Cada proceso altera exclusivamente su propia variable p.

d.

Si `p++` se coloca dentro del bucle `for` antes del `fork()` : El proceso padre incrementa p a 1 en la primera iteración antes de crear al primer hijo. Por lo tanto, ese hijo hereda p = 1. En la siguiente iteración, ambos incrementan p a 2 antes de hacer `fork()`. Al salir del bucle, los procesos imprimirán valores acumulados mayores (p = 3), ya que los hijos heredan el valor actualizado que poseía el padre en el instante preciso de la duplicación.

21. Dado un esquema de segmentación donde cada dirección hace referencia a 1 byte y la siguiente tabla de segmentos de un proceso, traduzca, de corresponder, las direcciones lógicas indicadas a direcciones físicas. La Dirección lógica está representada por: segmento: desplazamiento

Segmento	Dir. Base	Tamaño
0	102	12500
1	28699	24300
2	68010	15855
3	80001	400

- | | | | |
|------|------------|-----|------------|
| i. | 0000:9001 | iv. | 0001:18976 |
| ii. | 0001:24301 | v. | 0003:0 |
| iii. | 0002:5678 | | |

- i. **0000:9001**
Segmento 0, Desplazamiento 9001
¿9001 < 12500? Si
Dirección física = 102 + 9001 = 9103
- ii. **0001:24301**
Segmento 1, Desplazamiento 24301
¿24301 < 24300? No
Resultado = Fallo de segmento / Dirección invalida (trap)
- iii. **0002:5678**
Segmento 2, Desplazamiento 5678
¿5678 < 15855? Si
Dirección física = 68010 + 5678 = 73688
- iv. **0001:18976**
Segmento 1, Desplazamiento 18976
¿18976 < 24300? Si
Dirección física = 28699 + 18976 = 47675
- v. **0003:0**
Segmento 3, Desplazamiento 0
¿0 < 400? Si
Dirección física = 80001 + 0 = 80001

22. Suponga un sistema donde la memoria es administrada mediante la técnica de paginación, y donde:

- El tamaño de la página es de 512 bytes
- Cada dirección de memoria referencia 1 byte.
- Se tiene una memoria principal de 10240 bytes (10 KiB) y los marcos se enumeran desde 0 y comienzan en la dirección física 0.

Suponga además un proceso P1 con un tamaño lógico de 2000 bytes y con la siguiente tabla de páginas:

Página	Marco
0	3
1	5
2	2
3	6

a. Realice los gráficos necesarios (de la memoria principal, del espacio de direcciones del proceso, de tabla de páginas y de la tabla de marcos) en el que refleje el estado descrito.

b. Indicar si las siguientes direcciones lógicas corresponden al espacio lógico del proceso P1 y en caso afirmativo indicar la dirección física a la que corresponden:

- | | | | |
|------|------|-----|------|
| i. | 35 | iv. | 0 |
| ii. | 512 | v. | 1325 |
| iii. | 2051 | vi. | 602 |

c. Indicar, en caso de ser posible, las direcciones lógicas del proceso P1 que se corresponden a las siguientes direcciones físicas:

- | | | | |
|------|------|-----|------|
| i. | 509 | iv. | 3215 |
| ii. | 1500 | v. | 1024 |
| iii. | 0 | vi. | 2000 |

a.

1.Espacio de direcciones lógicas (P1):

+-----+ 0
Página 0 (512 B)
+-----+ 512
Página 1 (512 B)
+-----+ 1024
Página 2 (512 B)
+-----+ 1536
Página 3 (464 B +
48 B frag. interna)
+-----+ 2048 (Lógico: 2000 B)

2.Tabla de páginas (P1):

Página	Marco
0	3
1	5
2	2
3	6

3.Memoria Principal y Tabla de Marcos:

Representación de los 20 marcos físicos de la RAM y la ubicación no contigua de P1:

Marco	Dir. Física	Ocupación / Estado
19	9728 - 10239	Libre / Otro proceso
...	...	...
7	3584 - 4095	Libre / Otro proceso
6	3072 - 3583	Ocupado por P1 -> Página 3
5	2560 - 3071	Ocupado por P1 -> Página 1
4	2048 - 2559	Libre / Otro proceso
3	1536 - 2047	Ocupado por P1 -> Página 0
2	1024 - 1535	Ocupado por P1 -> Página 2
1	512 - 1023	Libre / Otro proceso
0	0 - 511	Libre / Otro proceso
0 B		

b.

I. 35

¿Pertenece a P1 ($35 < 2000$)? Si

Página: $35 // 512 = 0$

Desplazamiento: $35 \bmod 512 = 35$

Marco asignado: Página 0 → Marco 3

Dirección física: $(3 \times 512) + 35 = 1536 + 35 = 1571$

II. 512

¿Pertenece a P1 ($512 < 2000$)? Si

Página: $512 // 512 = 1$

Desplazamiento: $512 \bmod 512 = 0$

Marco asignado: Página 1 → Marco 5

Dirección física: $(5 \times 512) + 0 = 2560$

III. 2051

¿Pertenece a P1 ($2051 < 2000$)? No

IV. 0

¿Pertenece a P1 ($0 < 2000$)? Si

Página: $0 // 512 = 0$

Desplazamiento: $0 \bmod 512 = 512$

Marco asignado: Pagina 0 → Marco 3

Dirección física: $(3 \times 512) + 0 = 1536$

V. 1325

¿Pertenece a P1 ($1325 < 2000$)? Si

Página: $1325 // 512 = 2$

Desplazamiento: $1325 \bmod 512 = 301$

Marco asignado: Pagina 2 → Marco 2

Dirección física: $(2 \times 512) + 301 = 1024 + 301 = 1325$

VI. 602

¿Pertenece a P1 ($602 < 2000$)? Si

Página: $602 // 512 = 1$

Desplazamiento: $602 \bmod 512 = 90$

Marco asignado: Pagina 1 → Marco 3

Dirección física: $(3 \times 512) + 90 = 2560 + 90 = 2650$

c.

I. 509

Marco: $502 // 512 = 0$

Marco 0 no perteneciente a P1

II. 1500

Marco: $1500 // 512 = 2$

Desplazamiento: $1500 \bmod 512 = 476$

Página: Marco 2 → Página 2

Dirección lógica: $(2 \times 512) + 476 = 1024 + 476 = 1500$

III. 0

Marco: $0 // 512 = 0$

Marco 0 no perteneciente a P1

IV. 3215

Marco: $3215 // 512 = 6$

Desplazamiento: $3215 \bmod 512 = 143$

Página: Marco 6 → Página 3

Dirección lógica: $(3 \times 512) + 143 = 1536 + 143 = 1679$

V. 1024

Marco: $1024 // 512 = 2$

Desplazamiento: $1024 \bmod 512 = 0$

Página: Marco 2 $\rightarrow$ Página 2

Dirección lógica: $(2 \times 512) + 0 = 1024$

VI. 2000

Marco: $2000 // 512 = 3$

Desplazamiento: $2000 \bmod 512 = 464$

Página: Marco 3 $\rightarrow$ Página 0

Dirección lógica: $(0 \times 512) + 464 = 0 + 464 = 464$

23. Dado un esquema donde cada dirección hace referencia a 1 byte, con páginas de 2 KiB (KibiBytes), donde el frame 0 se encuentra en la dirección física 0. Con las siguientes primeras entradas de la tabla de páginas de un proceso, traduzca las direcciones lógicas indicadas a direcciones físicas:

i. 5120

ii. 3242

iii. 1578

iv. 2048

v. 8191

Página	Marco
0	16
1	13
2	9
3	2
4	0

i. **5120**

$$\text{Página} = 5120 // 2048 = 2$$

$$\text{Desplazamiento} = 5120 \bmod 2048 = 1024$$

$$\text{Marco asignado} = \text{Página } 2 \rightarrow \text{Marco } 9$$

$$\text{Dirección física} = (9 \times 2048) + 1024 = 18432 + 1024 = 19456$$

ii. **3242**

$$\text{Página} = 3242 // 2048 = 1$$

$$\text{Desplazamiento} = 3242 \bmod 2048 = 1194$$

$$\text{Marco asignado} = \text{Página } 1 \rightarrow \text{Marco } 13$$

$$\text{Dirección física} = (13 \times 2048) + 1194 = 26624 + 1194 = 27818$$

iii. **1578**

$$\text{Página} = 1578 // 2048 = 0$$

$$\text{Desplazamiento} = 1578 \bmod 2048 = 1578$$

$$\text{Marco asignado} = \text{Página } 0 \rightarrow \text{Marco } 16$$

$$\text{Dirección física} = (16 \times 2048) + 1578 = 32768 + 1578 = 34346$$

iv. **2048**

$$\text{Página} = 2048 // 2048 = 1$$

$$\text{Desplazamiento} = 2048 \bmod 2048 = 0$$

$$\text{Marco asignado} = \text{Página } 1 \rightarrow \text{Marco } 13$$

$$\text{Dirección física} = (13 \times 2048) + 0 = 26624$$

v. **8191**

$$\text{Página} = 8191 // 2048 = 3$$

$$\text{Desplazamiento} = 8191 \bmod 2048 = 2047$$

$$\text{Marco asignado} = \text{Página } 3 \rightarrow \text{Marco } 2$$

$$\text{Dirección física} = (2 \times 2048) + 2047 = 4096 + 2047 = 6143$$

24. Dado un esquema de paginación, donde cada dirección hace referencia a 1 byte, se tiene un tamaño de dirección de 32 bits y un tamaño de página de 2048 bytes. Suponga además un proceso P1 que necesita 51358 bytes para su código y 68131 bytes para sus datos.

- a. De los 32 bits de las direcciones ¿Cuántos se utilizan para identificar página? ¿Cuántos para desplazamiento?
- b. ¿Cuál sería el tamaño lógico máximo de un proceso?
- c. ¿Cuántas páginas máximo puede tener el proceso P1?
- d. Si se contaran con 4GiB (Gibibytes) de RAM, ¿cuántos marcos habría disponibles para utilizar?
- e. ¿Cuántas páginas necesita P1 para almacenar su código?
- f. ¿Cuántas páginas necesita P1 para almacenar sus datos?
- g. ¿Cuántos bytes habría de fragmentación interna entre lo necesario para almacenar su código y sus datos? Ayuda: Recuerde que en una misma página no pueden convivir código y datos.
- h. Asumiendo que el hardware utiliza tabla de páginas de un (1) nivel, que la dirección se interpreta como 20 bits para identificar página y 12 bits para desplazamiento, y que cada Entrada de Tabla de Páginas (PTE) requiere de 1 Kib. ¿Cuál será el tamaño mínimo que la tabla de páginas del proceso P1 ocupará en RAM?
- i. Asumiendo que el hardware utiliza tabla de páginas de dos (2) niveles, que la dirección se interpreta como 10 bits para identificar primer nivel, 10 bits para identificar el segundo nivel, 12 bits para desplazamiento y que cada Entrada de Tabla de Páginas (PTE) requiere de 1 Kib. ¿Cuál será el tamaño mínimo que la tabla de páginas del proceso P1 ocupará en RAM?
- j. Dado los resultados obtenidos en los dos incisos anteriores (h, i) ¿Qué conclusiones se pueden obtener del uso de los 2 modelos de tablas de página?

a.

- **Desplazamiento (Offset):** $\log_2(2048) = 11$ bits.
- **Identificador de Página:** $32 - 11 = 21$ bits.

b.

Con direcciones de 32 bits direccionables por byte:

$$2^{32} \text{ bytes} = 4.294.967.296 \text{ bytes} = 4 \text{ GiB}$$

c.

Al disponer de 21 bits para el número de página en su espacio de direcciones virtuales, el máximo de páginas direccionables por un proceso es:

$$2^{21} = 2.097.152 \text{ páginas}$$

d.

Dividiendo la memoria principal entre el tamaño de página:

Marcos = $4 \text{ GiB} / 2048 \text{ bytes} = 4.294.967.296 \text{ bytes} / 2048 \text{ bytes} = 2.097.152 \text{ marcos}$

e.

Páginas de código = $[51358 \text{ bytes} / 2048 \text{ bytes}] = [25,077] = 26 \text{ paginas}$

f.

Páginas de datos = $[68131 \text{ bytes} / 2048 \text{ bytes}] = [33,267] = 34 \text{ paginas}$

g.

Como código y datos no pueden convivir en la misma página:

- **Sección de código:**

Asignado = $26 \times 2048 = 53.248 \text{ bytes}$.

Fragmentación = $53.248 - 51.358 = 1.890 \text{ bytes}$.

- **Sección de datos:**

Asignado = $34 \times 2048 = 69.632 \text{ bytes}$.

Fragmentación = $69.632 - 68.131 = 1.501 \text{ bytes}$.

- **Fragmentación interna total:**

$1.890 + 1.501 = 3.391 \text{ bytes}$

h.

En un esquema de 1 nivel con 20 bits de página, la tabla de páginas debe contener entradas para todas las páginas posibles ($2^{20} = 1.048.576$ entradas). Si cada entrada (PTE) ocupa **1 KiB**:

Tamaño = $1.048.576 \text{ PTEs} \times 1 \text{ KiB} = 1.048.576 \text{ KiB} = 1024 \text{ MiB} = 1 \text{ GiB}$

i.

Nivel 1 (Directorio raíz): 10 bits $\rightarrow 2^{10} = 1.024$ entradas.

Ocupa: $1.024 \times 1 \text{ KiB} = 1 \text{ MiB}$.

Nivel 2: Cada tabla de segundo nivel posee $2^{10} = 1.024$ entradas y abarca un rango de 1.024 páginas.

Proceso P1: Requiere solo 60 páginas en total (26 de código y 34 de datos).

- **Caso Mínimo (Código y datos en el mismo bloque/entrada de Nivel 1):** Se necesita 1 Tabla de Nivel 1 (1 MiB) + 1 Tabla de Nivel 2 (1 MiB) = 2 MiB (2.048 KiB).
- **Caso Mínimo con separación de regiones (Código y datos en entradas distintas de Nivel 1):** Se necesita 1 Tabla de Nivel 1 (1 MiB) + 2 Tablas de Nivel 2 (2 MiB) = 3 MiB (3.072 KiB).

j.

- **Paginación de 1 Nivel (Monolítica):** Es extremadamente ineficiente en el uso de memoria. Fuerza al sistema operativo a reservar en RAM un bloque contiguo masivo de 1 GiB únicamente para la tabla de páginas, donde el 99.9% de las entradas están vacías porque P1 solo usa 60 páginas.
- **Paginación de 2 Niveles (Multinivel):** Resuelve el problema instanciando en RAM exclusivamente la tabla raíz y las tablas de segundo nivel que corresponden a páginas efectivamente utilizadas. Reduce el consumo en RAM para las tablas de 1 GiB a solo 2 MiB (o 3 MiB), haciendo viable la administración de memoria en arquitecturas con espacios lógicos amplios.

25. Dada la técnica de administración de memoria por medio de segmentación paginada y teniéndose disponibles las siguientes tablas:

Tabla de Segmentos		Tabla de Páginas		
Núm. Seg.	Dir. base	Nro. Segmento	Nro. Página	Direc. Base
1	500	1	1	40
			2	80
			3	60
2	1500	2	1	20
			2	25
			3	0
3	5000	3	1	120
			2	150

Indicar las direcciones físicas correspondientes a las siguientes direcciones lógicas (segmento,página,desplazamiento):

i. (2,1,1)

ii. (1,3,15)

iii. (3,1,10)

iv. (2,3,5)

i. **(2,1,1)**

Segmento 2 | Pagina 1 | Desplazamiento 1

Dirección base = 20

Dirección física = $20 + 1 = 21$

ii. **(1,3,15)**

Segmento 1 | Pagina 3 | Desplazamiento 15

Dirección base = 60

Dirección física = $60 + 15 = 75$

iii. **(3,1,10)**

Segmento 3 | Pagina 1 | Desplazamiento 10

Dirección base = 120

Dirección física = $120 + 10 = 130$

iv. **(2,3,5)**

Segmento 2 | Pagina 3 | Desplazamiento 5

Dirección base = 0

Dirección física = $0 + 5 = 5$